CodeSphere AI: An AI-Augmented Software Engineering Collaboration Platform

Project Proposal for UCS503P
Submitted to: Ms. Paramveer Sidhu
Thapar Institute of Engineering and Technology

Sarthak Soni, 3C11* Ayush Bansal, 3C11†

August 17, 2026

1 Higher-order goal

. . . secondary framing

This project aims to improve software engineering productivity by reducing the time developers spend understanding unfamiliar codebases, triaging issues, and coordinating work across a team. While the underlying theme (AI-assisted developer tooling) is broad, the immediate engineering objective is to translate *faster, better-informed engineering decisions* into a measurable, deployable software solution.

2 Time-to-value

. . . primary heuristic

- We will deliver meaningful increments quickly by prototyping the core repository-insight workflow and validating it against a real open-source repository within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback.

3 Problem Statement

Third-year CS teams and early-career developers routinely lose time on tasks that are procedural rather than creative: understanding what a codebase does, finding the right file/module for a bug, writing consistent documentation, and tracking who is doing what. Common pain points include:

- **Onboarding friction:** new contributors take long to understand project structure and module responsibilities.
- **Fragmented task tracking:** tasks/issues live separately from the code, so context is repeatedly re-explained.

*Roll No: 1024030061, Email: ssoni1_be24thapar.edu
†Roll No: 1024030059, Email: abansal9_be24thapar.edu

- **Manual, inconsistent code review:** reviewers spend time on style/documentation nits instead of logic and design.
- **Poor visibility:** team leads lack a single view of project health (open issues, stale PRs, module ownership).

Why this matters now (contextual relevance): Generative and retrieval-based AI models are mature and cheap enough to embed directly into everyday developer workflows; a lightweight integration layer can meaningfully cut down repetitive engineering overhead even for small teams.

4 Proposed Solution *... Software Proposition*

4.1 Overview

Build a **web-based** "CodeSphere AI" dashboard, backed by a **GitHub App/webhook bot**, that adds AI-assisted codebase understanding on top of GitHub rather than replacing GitHub's own project-management features. We deliberately do *not* build a custom Kanban/task board – GitHub Projects already covers that – and instead surface GitHub's existing issues in our dashboard with an AI layer on top. Components:

- **Repository connector:** link a GitHub repository via OAuth (read-only) and index its structure, commits, and open issues.
- **AI codebase assistant (website):** chat interface that answers questions about the codebase ("where is X handled?", "summarize this module") using retrieval over indexed source files, with citations to file/line.
- **Auto-triage bot (GitHub App/webhook):** on every new issue/PR, the bot posts a generated summary and a suggested label as a comment directly on GitHub – this is the feature GitHub does not provide natively.
- **Team dashboard (website):** read-only view of GitHub issues/PRs enriched with staleness alerts and module ownership, pulled live from the GitHub API (not a separate board).

4.2 Core workflow

1. Team lead connects a GitHub repository; system indexes files, commit history, and open issues.
2. Contributor asks the AI assistant a question about the codebase (on the website) and receives an answer grounded in retrieved source snippets, with file/line references.
3. When a new issue/PR is opened on GitHub, the webhook bot auto-comments with a summary and suggested label within seconds.
4. Team lead views the dashboard for backlog health, stale items, and module ownership, sourced live from GitHub – no data duplicated into a separate board.

4.3 Operational constraints for an educational, third-year setting

- Keep the solution usable with a single GitHub OAuth connection and no heavyweight infrastructure.
- Avoid dependence on training custom ML models; focus on retrieval, orchestration, and workflow correctness using existing LLM APIs.
- Design for deployability using common web hosting and managed databases.

5 Solution Approach *... Engineering focus*

This is an engineering course project, so we focus on scalable integration, retrieval, and workflow aspects rather than novel model research.

5.1 Codebase understanding *... non-research*

Code intelligence will be retrieval-based, not model-training-based:

- Chunk source files by function/class boundaries and generate embeddings using an existing embedding API.
- Store embeddings in a vector index; retrieve top-k relevant chunks for a given question.
- Pass retrieved chunks plus the question to an LLM API to generate a grounded answer with citations to file paths; never claim state-of-the-art accuracy.

5.2 Web architecture *... web-based solution*

Two cooperating pieces, sharing one backend and database:

- **Website (frontend + backend API):** responsive UI for the chat assistant and dashboard; backend handles GitHub OAuth, repository indexing jobs, and LLM orchestration.
- **GitHub App / webhook service:** a small service registered as a GitHub App on the repository; GitHub pushes "issue opened" / "PR opened" events to it, it calls the same LLM orchestration layer, and posts the summary/label back to GitHub via the GitHub API.
- **Database:** store users, repositories, indexed chunks/embeddings, and generated summaries (issues/tasks themselves stay in GitHub – we do not duplicate that data).

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features.

6 Evaluation Criterion *... measurable and attributable*

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Answer grounding rate: percentage of AI-assistant answers that correctly cite an existing, relevant file/line in the indexed repository.

- Target: $\geq 85\%$ grounding rate on a held-out set of 30 test questions during pilot.
- Attribution: measured by comparing cited file references against a manually verified answer key.

6.2 Secondary evaluation metrics

- **Time-to-first-understanding:** time for a new contributor to locate the correct module for a sample task, with vs. without the assistant.
- **Issue triage accuracy:** percentage of auto-generated labels/summaries accepted without edit by the contributor.
- **Triage latency:** time between an issue being opened on GitHub and the bot posting its summary comment (target: under 30 seconds).
- **Reliability:** availability of staging/production for login and repository indexing during business hours (e.g., $\geq 99\%$ uptime in pilot).

6.3 Pilot validation plan *... high-level*

- Recruit 2–3 small student project teams (5–8 contributors total) and index one real repository per team.
- Compare onboarding/triage metrics before/after within the iteration window by logging baseline estimates via short interviews or manual timing.

7 Scalability *... with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in repository size and number of concurrent teams by:
   - decoupling components (frontend/backend/indexing worker),
   - incremental re-indexing on new commits instead of full re-index,
   - using stateless API design and a dedicated vector index for retrieval.
2. **Operationally deployable (practically):** The system should run on common web hosting:
   - managed relational database plus a managed vector store,
   - containerized or platform-hosted deployment,
   - CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project:

- GitHub API and OAuth for repository access and authentication.
- Web framework for REST APIs and reactive pages.
- Managed relational database plus a managed vector database for embeddings.
- Existing LLM and embedding APIs for summarization, Q&A, and retrieval.
- Test framework for unit/integration tests.
- CI runner and staging deployment target.

We will use these mature components to focus on workflow design, retrieval correctness, and measurement rather than inventing new infrastructure or training new models.

9 Project scope and deliverables *. . . iteration-friendly*

9.1 Initial deliverable *. . . first iteration*

- GitHub OAuth login + repository connection
- Repository indexing pipeline (chunking + embeddings) for one repository
- AI assistant chat with grounded, cited answers (website)
- GitHub App/webhook bot that auto-comments a summary on newly opened issues
- CI: build + backend unit tests + linting
- CD to staging with a single command or pipeline job

9.2 Subsequent deliverables

- Suggested-label posting alongside the summary (not just summary)
- Incremental re-indexing on new commits
- Team dashboard: stale-issue alerts and module ownership, pulled live from the GitHub API
- Multi-repository and multi-team support

10 Risks and mitigations

- **LLM cost/rate limits:** cache embeddings and answers; batch indexing jobs; set per-team usage quotas.
- **Answer hallucination:** always require retrieved-chunk citations; show "no confident answer" when retrieval confidence is low rather than guessing.
- **Repository access/privacy:** request minimal OAuth scopes; allow teams to disconnect and delete indexed data.
- **Evaluation measurement ambiguity:** fix the test question set and grounding-verification process before development begins.
- **Operational issues in pilot:** use staging early; ensure CI/CD produces repeatable deployments.

11 Summary

This project proposes a deployable web platform, CodeSphere AI, that reduces onboarding and coordination overhead for student and early-career software teams by grounding an AI assistant in a team's own codebase and linking it to a lightweight task workflow. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially answer grounding rate). It remains focused on engineering integration, retrieval correctness, and scalability readiness rather than research-driven model novelty.